

回顾

□ 图生成树

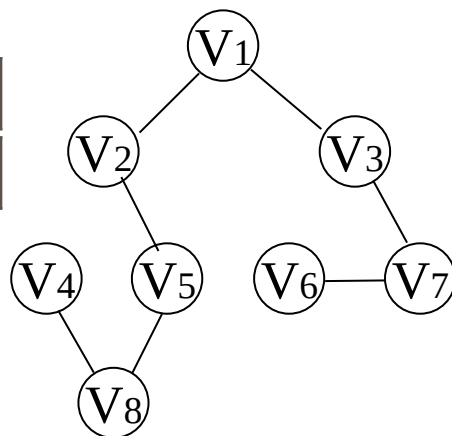
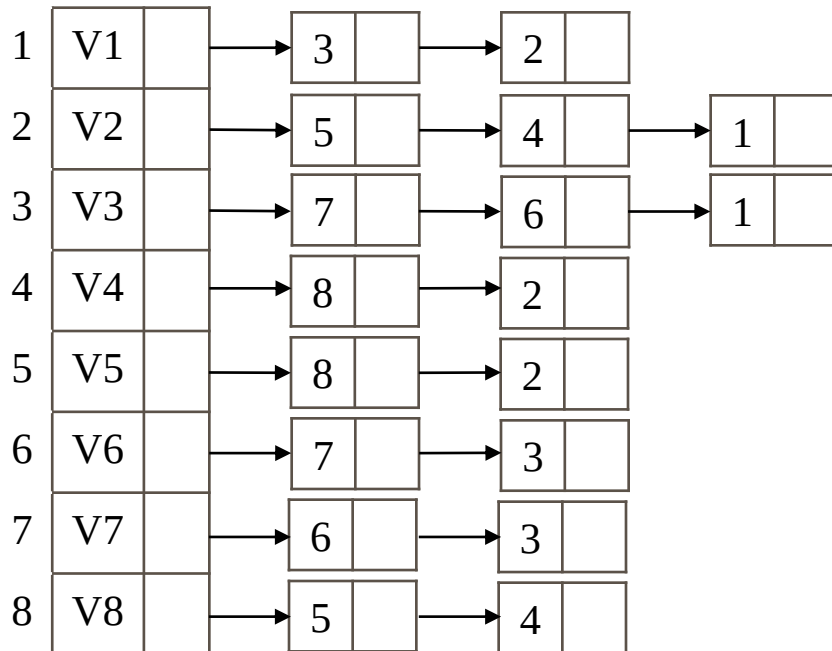
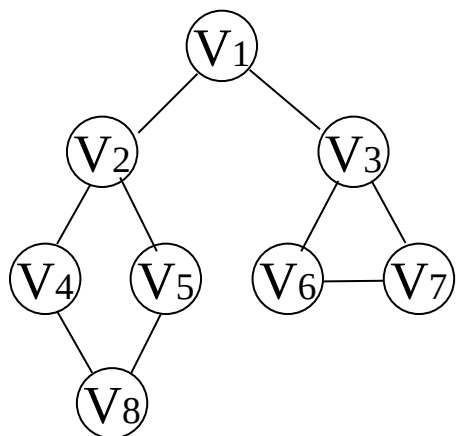
定义1 对于无向图连通 G 和一棵树 T 来说，如果 T 是 G 的生成子图，则称 T 是 G 的生成树。

- 生成树的顶点个数与图的顶点个数相同（是连通图的极小连通子图）
- 一个有 n 个顶点的连通图的生成树只有 $n-1$ 条边
- 一个图生成的树并不一定唯一
- 在生成树中再加一条边必然形成回路
- 生成树中任意两个顶点间的路径是唯一的

回顾

□ 图生成森林

非连通图可分解为多个连通分量，而每个连通分量又各自对应多个生成树，因此与整个非连通图相对应的，是由多棵生成树组成的生成森林。

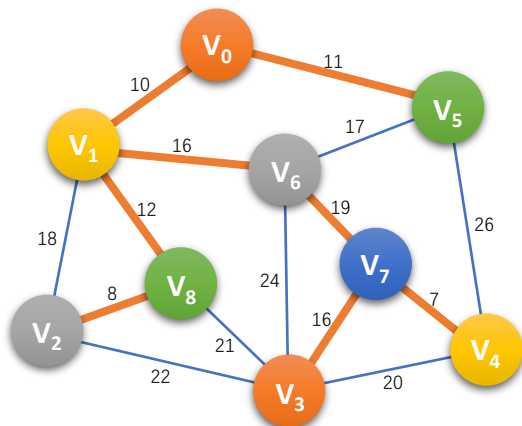


回顾

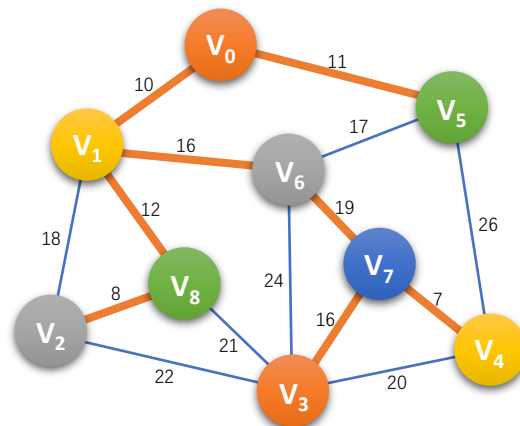
□ 最小生成树

在一个连通网的所有生成树中，各边的代价之和最小的那棵生成树称为该连通网的最小代价生成树。

- 连通网的**最小生成树的 $n-1$ 边中**一定包含连通网中**权值最小的边**。



Prim (普里姆) 算法



Kruskal (克鲁斯卡尔) 算法

第7章 图

7.1 图的定义和术语

7.2 图的存储结构

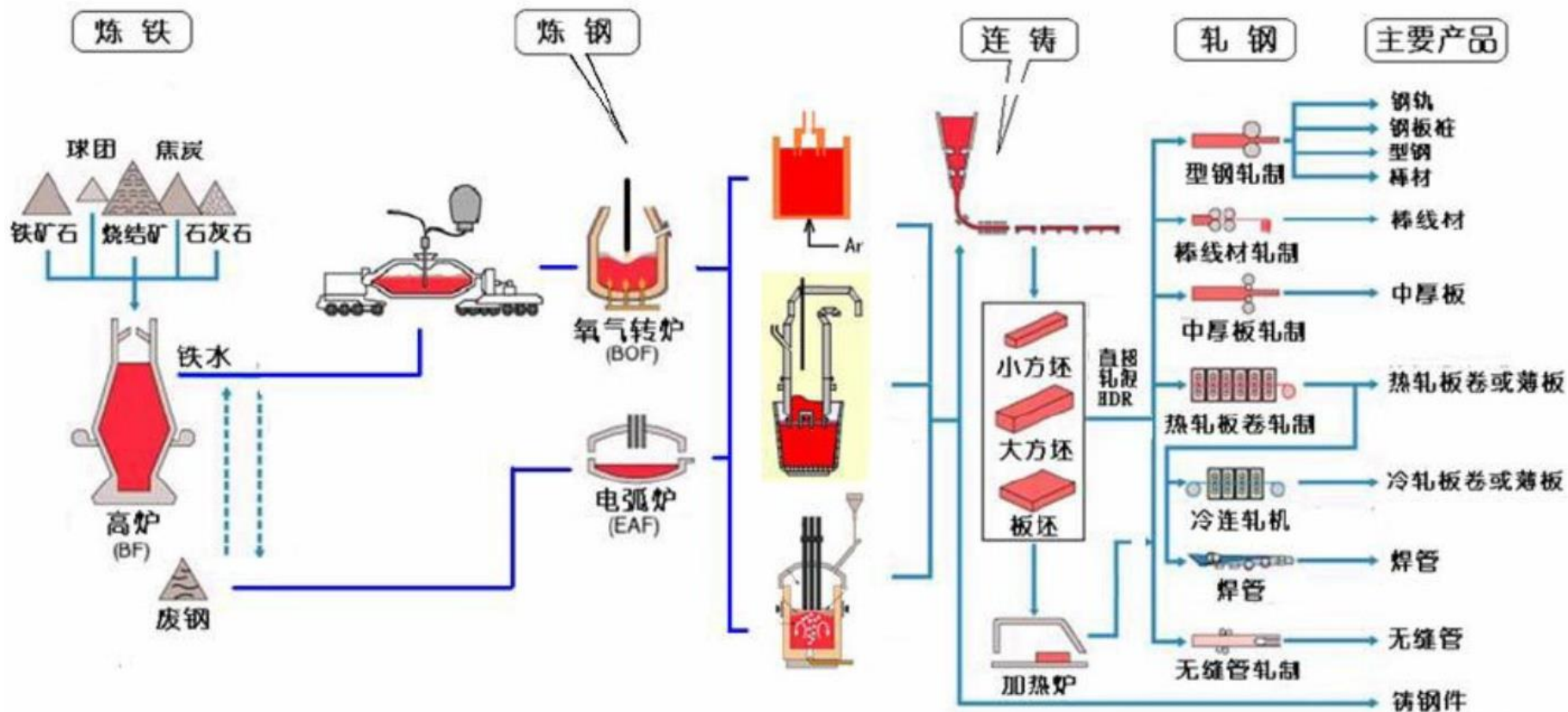
7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.6 最短路径

7.5 活动网络

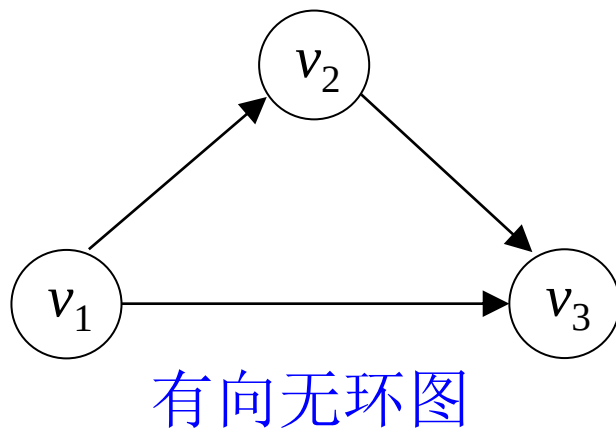
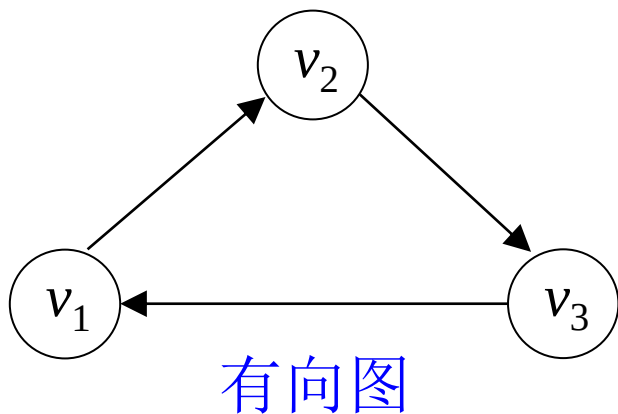


钢铁生产流程图

7.5 活动网络

在工程实践中，一个工程项目往往由若干个子项目组成，这些子项目间往往有多种关系：

- ①先后关系，即必须在一子项目完成后，才能开始实施另一个子项目；
- ②子项目之间无次序要求，即两个子项目可以同时进行，互不影响。



有向无环图(Directed acycline graph)简称DAG图，是描述一项工程或系统的进行过程的有效工具。

7.5 活动网络

□ 有向无环图

对整个工程和系统，关注两个方面的问题：

- ✓ 一是工程能否顺利进行；
- ✓ 二是估算整个工程完成所必须的最短时间。

用途：我们经常用有向图来描述一个工程或系统的进行过程。一般来说，一个工程可以分为若干个子工程，只要完成了这些子工程，就可以导致整个工程的完成。——**活动网络**

7.5 活动网络

□ 两种常用的活动网络(Activity Network)

① AOV网(Activity On Vertices)—用顶点表示活动的网络

AOV网定义：若用有向图表示一个工程，在图中用顶点表示活动，用弧表示活动间的优先关系。 v_i 必须先于活动 v_j 进行。则这样的有向图叫做用顶点表示活动的网络，简称 AOV。

② AOE网(Activity On Edges)—用边表示活动的网络

AOE网定义：如果在无环的带权有向图中，用有向边表示一个工程中的活动，用边上权值表示活动持续时间，用顶点表示事件，则这样的有向图叫做用边表示活动的网络，简称 AOE。

7.5 活动网络

□ 问题提出： **教学计划的制定**--**学生选修课程安排问题**：

哪些课程是必须先修的，哪些课程是可以并行学习的。 学生应按怎样的顺序学习这些课程，才能有组织、有步骤、顺利地地完成学业——**AOV网的拓扑排序**

➤ **顶 点**—表示**课程**

➤ **有向弧**—表示**先决条件**，若课程 v_i 是课程 v_j 的先决（先修）条件，则图中有弧 $\langle v_i, v_j \rangle$ 。

7.5 活动网络

□ AOV网络

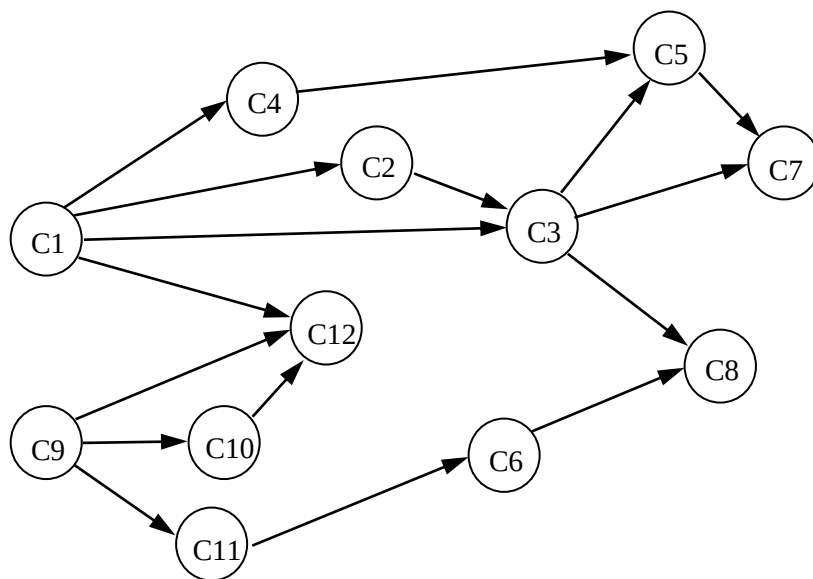
课程编号	课程名称	先导课程编号
C1	程序设计基础	无
C2	离散数学	C1
C3	数据结构	C1, C2
C4	汇编语言	C1
C5	算法分析与设计	C3, C4
C6	计算机组成原理	C11
C7	编译原理	C5, C3
C8	操作系统	C3, C6
C9	高等数学	无
C10	线性代数	C9
C11	普通物理	C9
C12	数值分析	C9, C10, C1

7.5 活动网络

□ AOV网络

学生选课问题

课程编号	课程名称	先导课程编号
C1	程序设计基础	无
C2	离散数学	C1
C3	数据结构	C1, C2
C4	汇编语言	C1
C5	算法分析与设计	C3, C4
C6	计算机组成原理	C11
C7	编译原理	C5, C3
C8	操作系统	C3, C6
C9	高等数学	无
C10	线性代数	C9
C11	普通物理	C9
C12	数值分析	C9, C10, C1



7.5.1 拓扑排序

□ 拓扑排序

对一个有向无环图中的顶点排成一个具有前后次序的线性序列。

拓扑排序算法：重复选择没有直接前驱的顶点。

7.5.1 拓扑排序

□ 拓扑排序

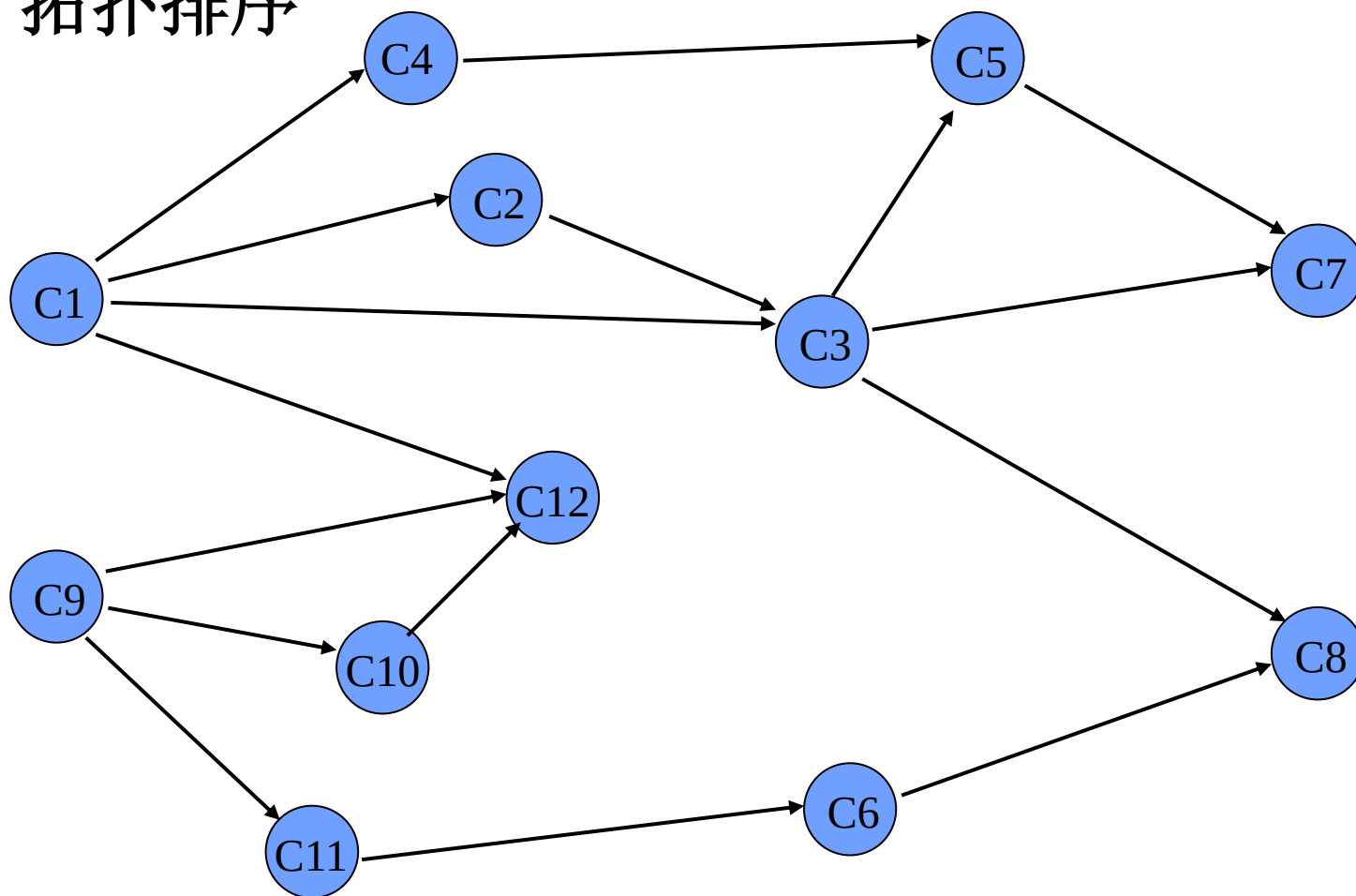
1. 输入AOV网络。令 n 为顶点个数。
2. 在AOV网络中选一个没有直接前驱的顶点, 并输出之;
3. 从图中删去该顶点, 同时删去所有它发出的有向边;
4. 重复以上 2、3 步, 直到:

全部顶点均已输出, 拓扑有序序列形成, 拓扑排序完成

或者, 图中还有未输出的顶点, 但已跳出处理循环。这说明图中还剩下一些顶点, 它们都有直接前驱, 再也找不到没有前驱的顶点了。这时AOV网络中必定存在有向环。

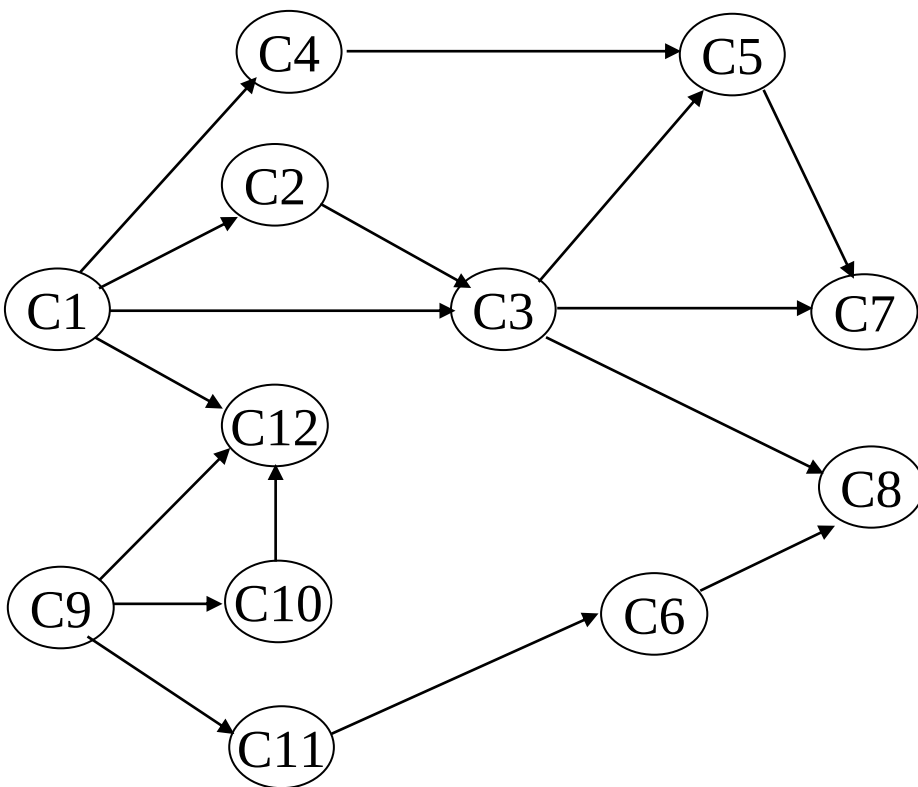
7.5 活动网络

□ 拓扑排序



拓扑序列: C1 C2 C3 C4 C5 C7 C9 C10 C11 C6 C12 C8

7.5.1 拓扑排序



★ 拓扑排序的方法

- ❖ 在有向图中选一个没有前驱的顶点并输出之
- ❖ 从图中删除该顶点和所有以它为尾的弧
- ❖ 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止

拓扑序列：C1--C2--C3--C4--C5--C7--C9--C10--C11--C6--C12--C8
或 ： C9--C10--C11--C6--C1--C12--C4--C2--C3--C5--C7--C8

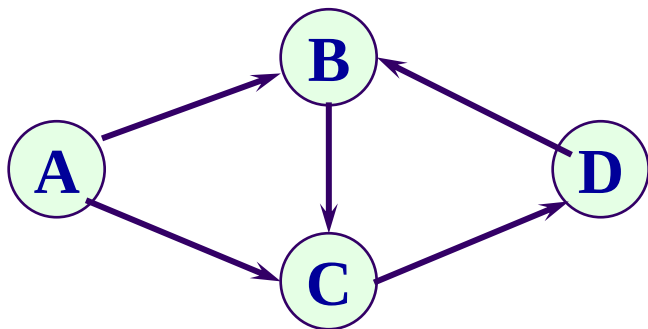
注：一个AOV网的拓扑序列**不是唯一的**

7.5.1 拓扑排序

□ 拓扑排序检测AOV网中是否存在环方法：

对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则该AOV网必定不存在环。

例如下列有向图



不能求得它的拓扑有序序列。

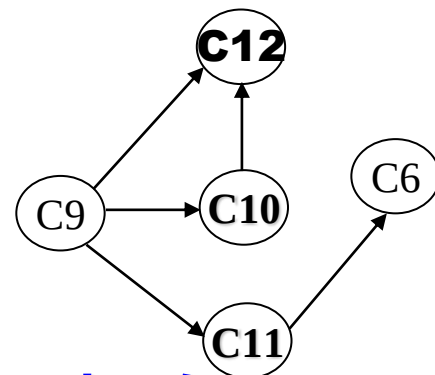
因为图中存在一个回路 {B, C, D}

7.5.1 拓扑排序

拓扑排序的算法实现

★ 拓扑排序的方法

1. 在有向图中选一个没有前驱的顶点并输出之
2. 从图中删除该顶点和所有以它为尾的弧
3. 重复上述两步，直至全部顶点均已输出；或者当图中不存在无前驱的顶点为止



☀ 在算法中需要用定量的描述替代定性的概念

✓ 没有前驱的顶点 = 入度为零的顶点

✓ 删除顶点及以它为尾的弧 = 弧头顶点的入度减1

❖ 宜以邻接表作存储结构

❖ 为便于顶点的入度减1，应增加一个存放顶点入度的数组indegree[n]。

7.5.1 拓扑排序

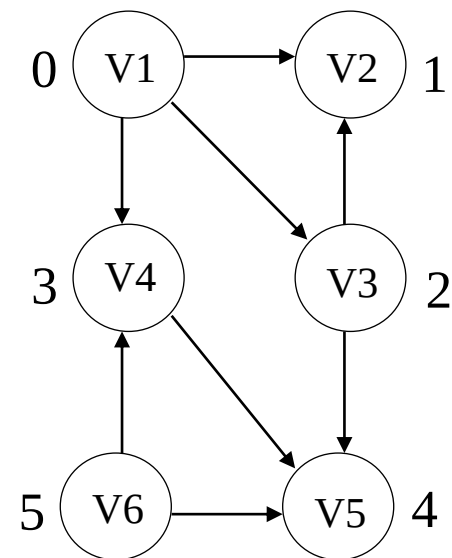
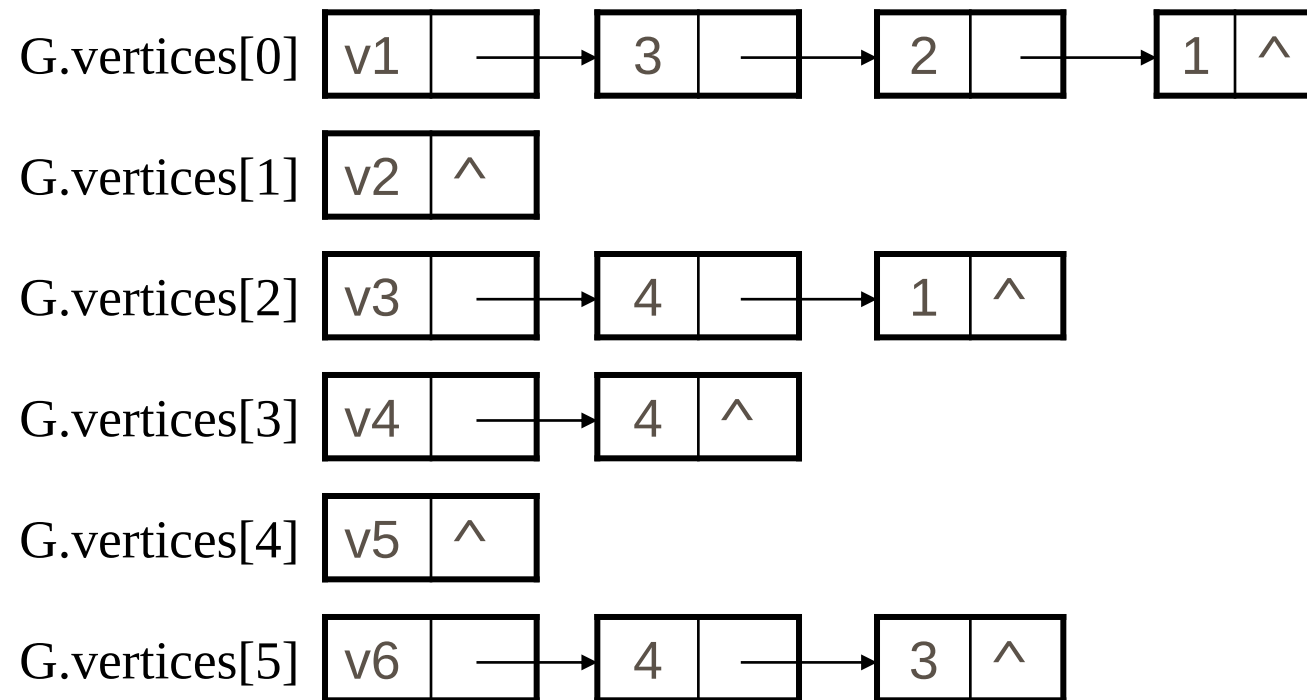
拓扑排序的算法实现（续）

为避免每次都要搜索入度为零的顶点，在算法中设置一个“栈”，以保存“入度为零”的顶点。

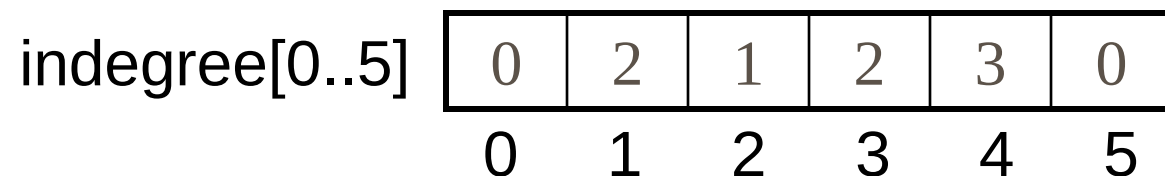
- 算法实现

- 求各顶点的入度 $\text{indegree}[0 \dots \text{vexnum}-1]$
- 初始化栈
- 把邻接表中所有入度为0的顶点进栈。
- 栈非空时，输出栈顶元素 V_j 并退栈；在邻接表中查找 V_j 的直接后继 V_k ，把 V_k 的入度减1；若 V_k 的入度为0则进栈。
- 重复上述操作直至栈空为止。若栈空时输出的顶点个数不是 n ，则有向图有环；否则，拓扑排序完毕。

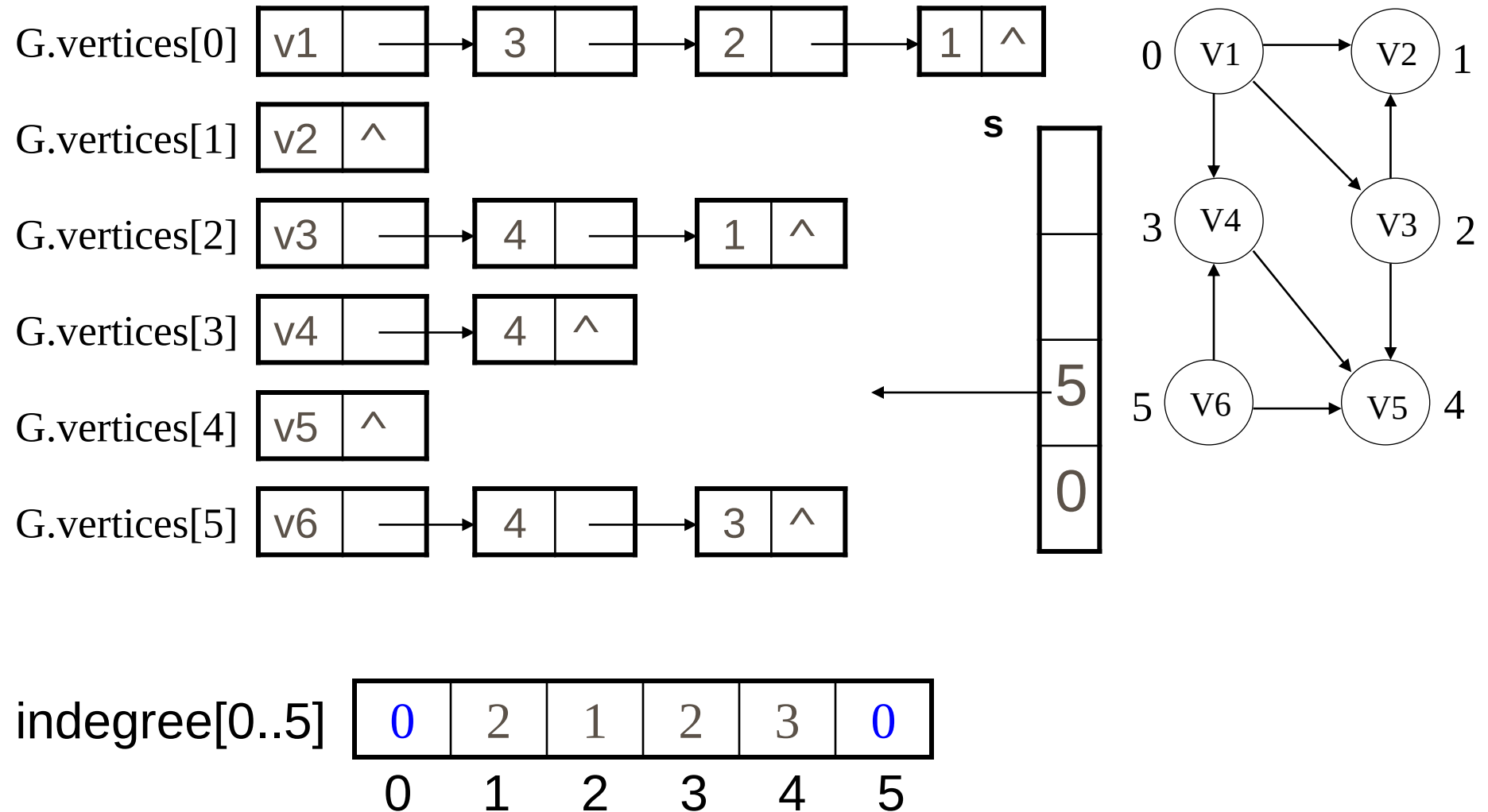
7.5.1 拓扑排序



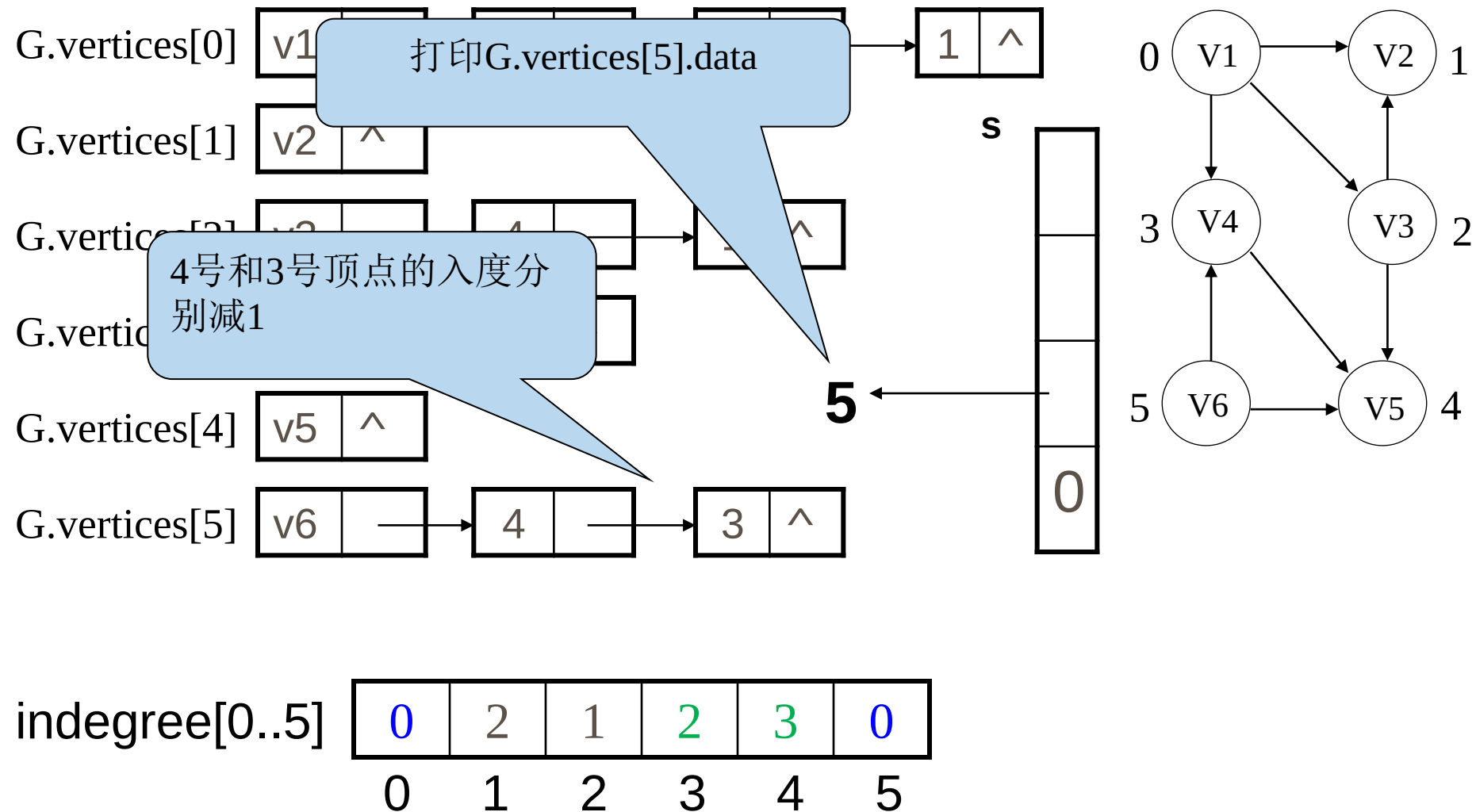
另外增设一个存放各顶点的入度值的一维数组 Indegree



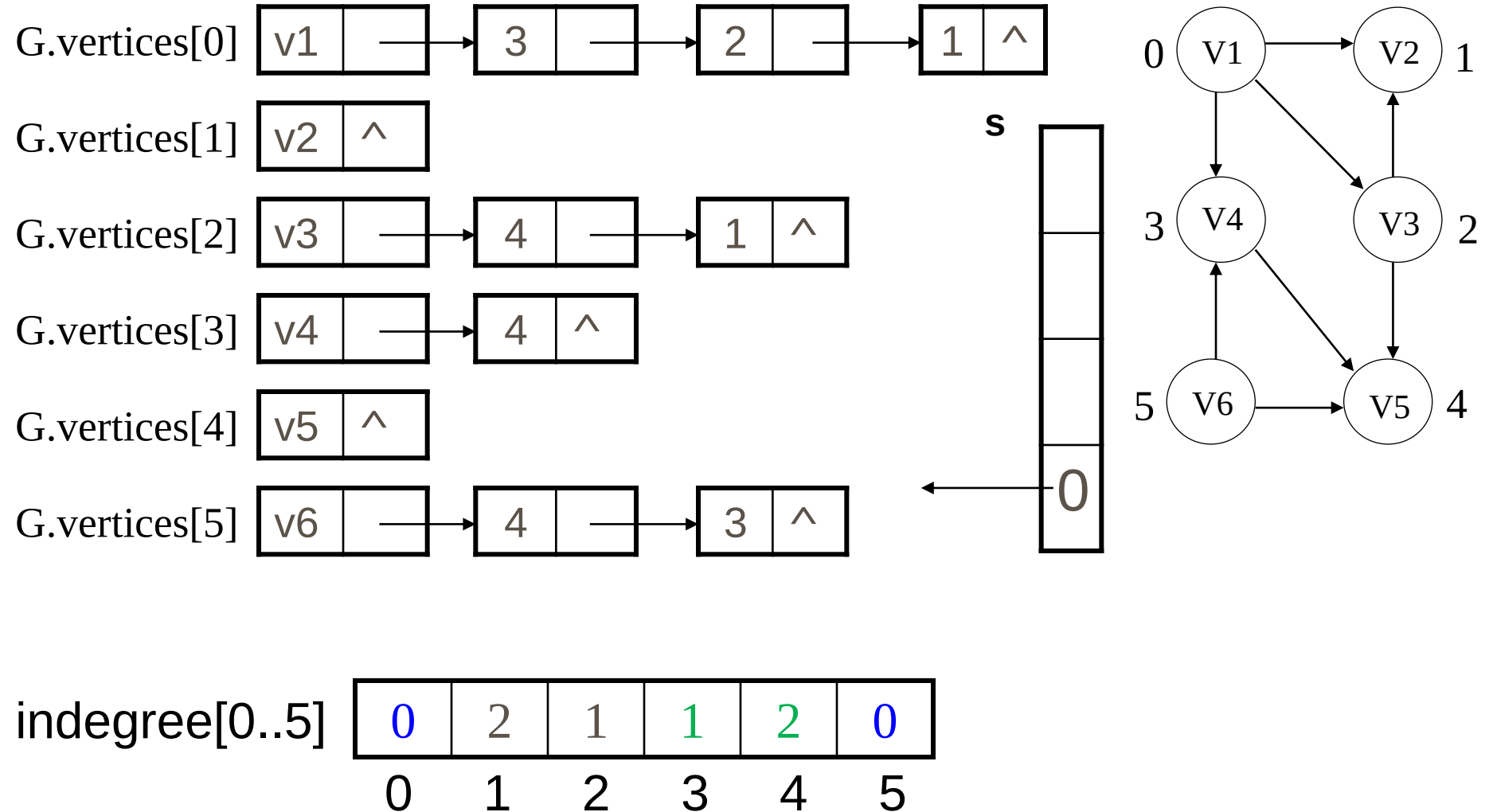
7.5.1 拓扑排序



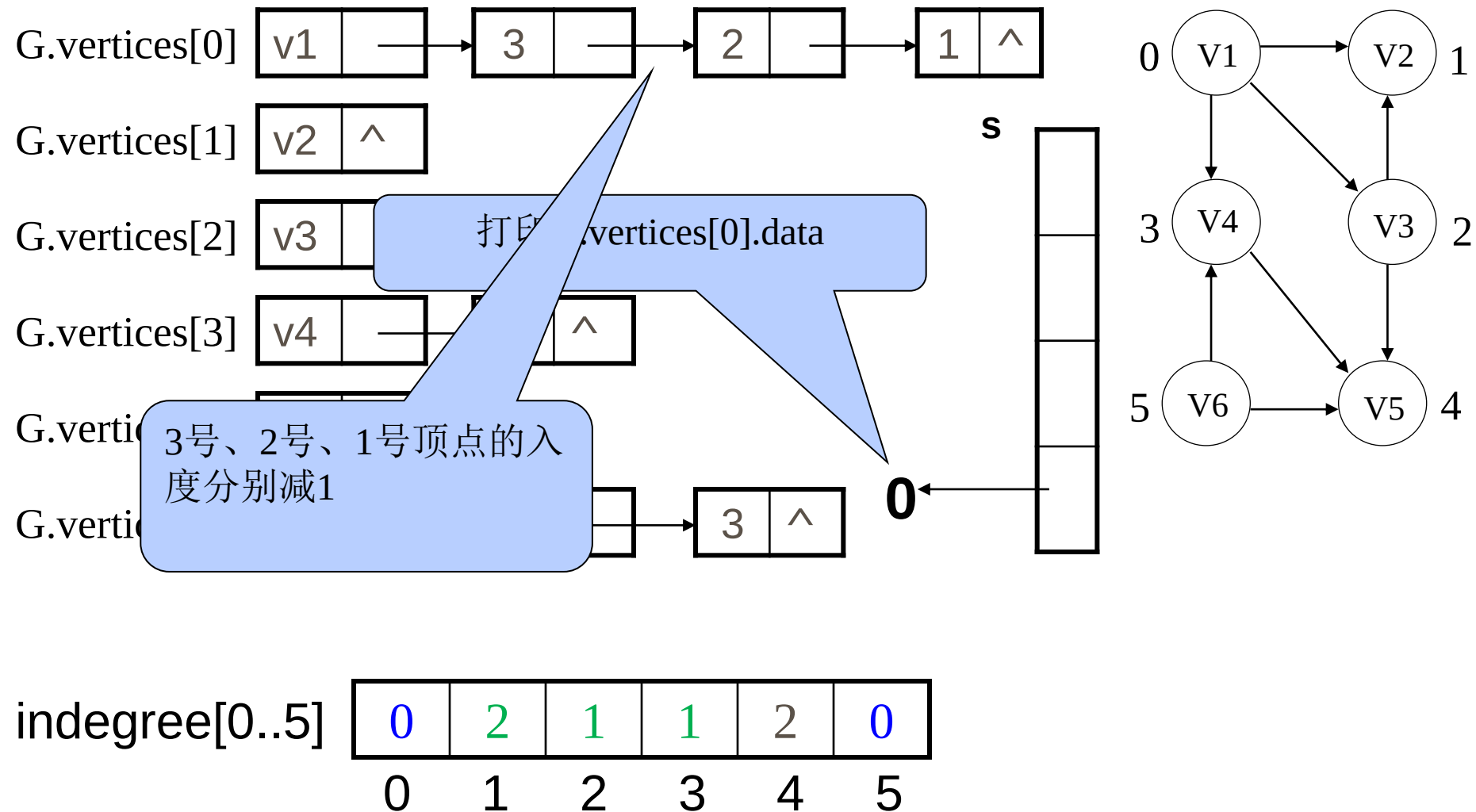
7.5.1 拓扑排序



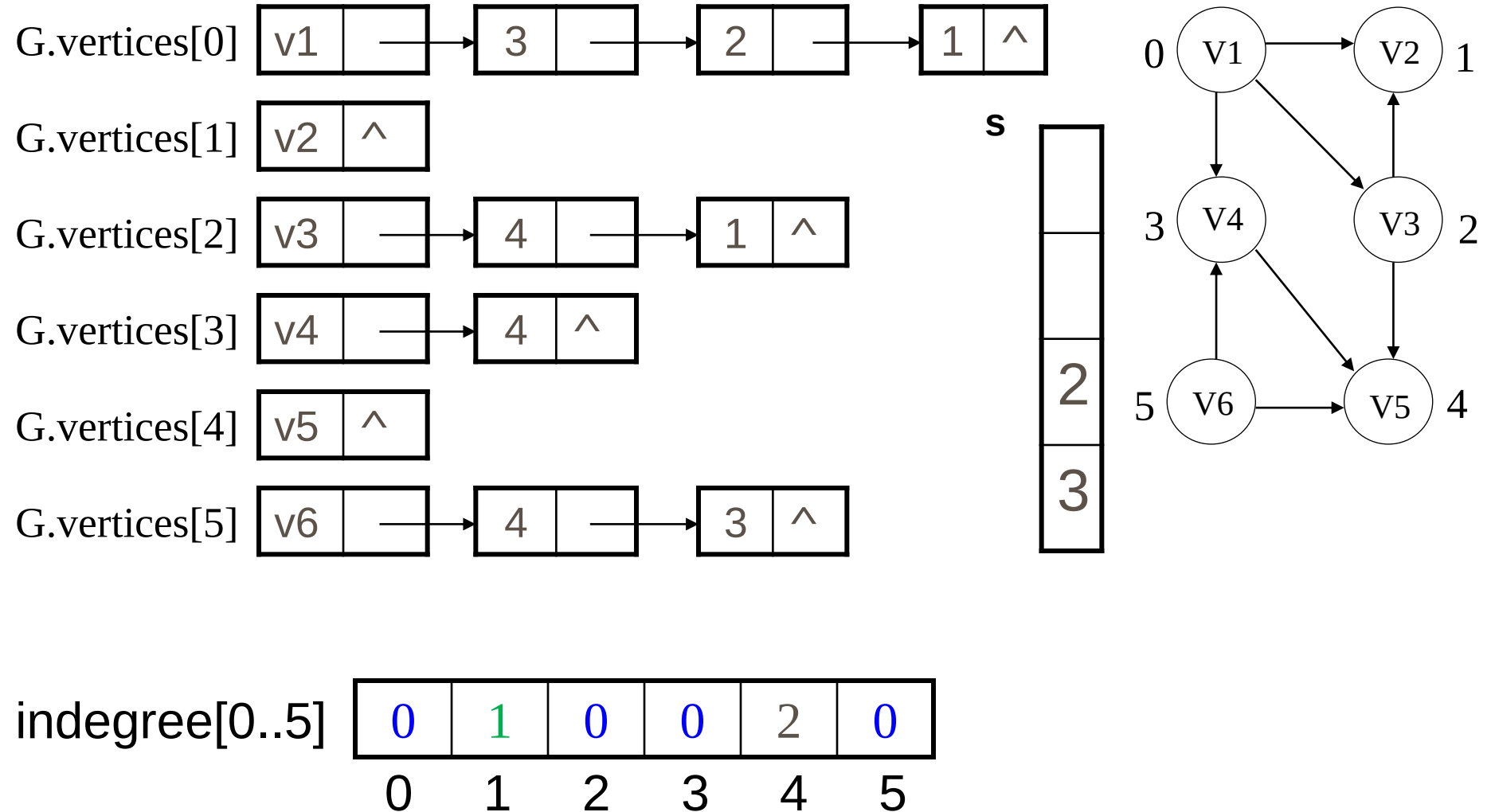
7.5.1 拓扑排序



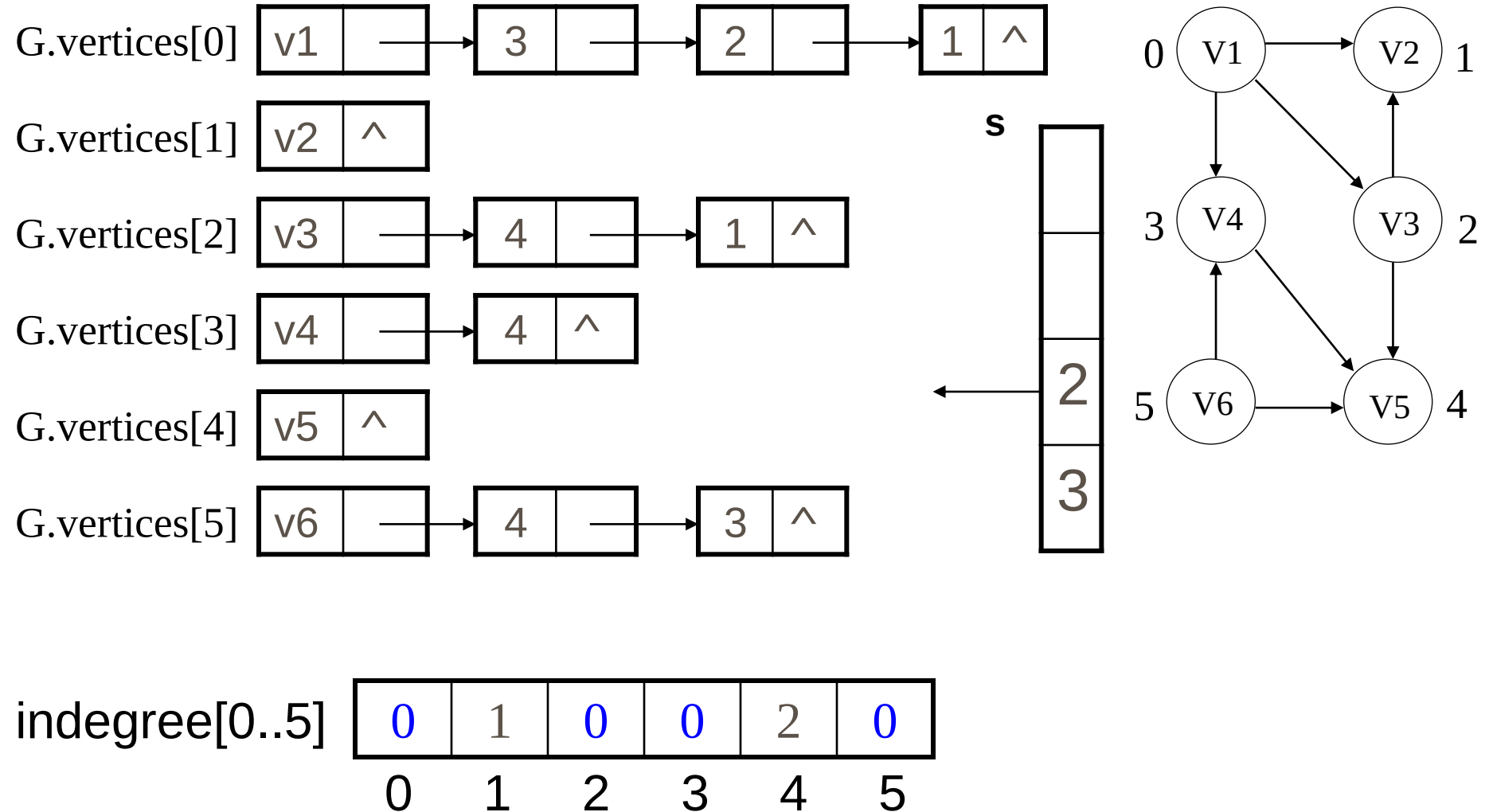
7.5.1 拓扑排序



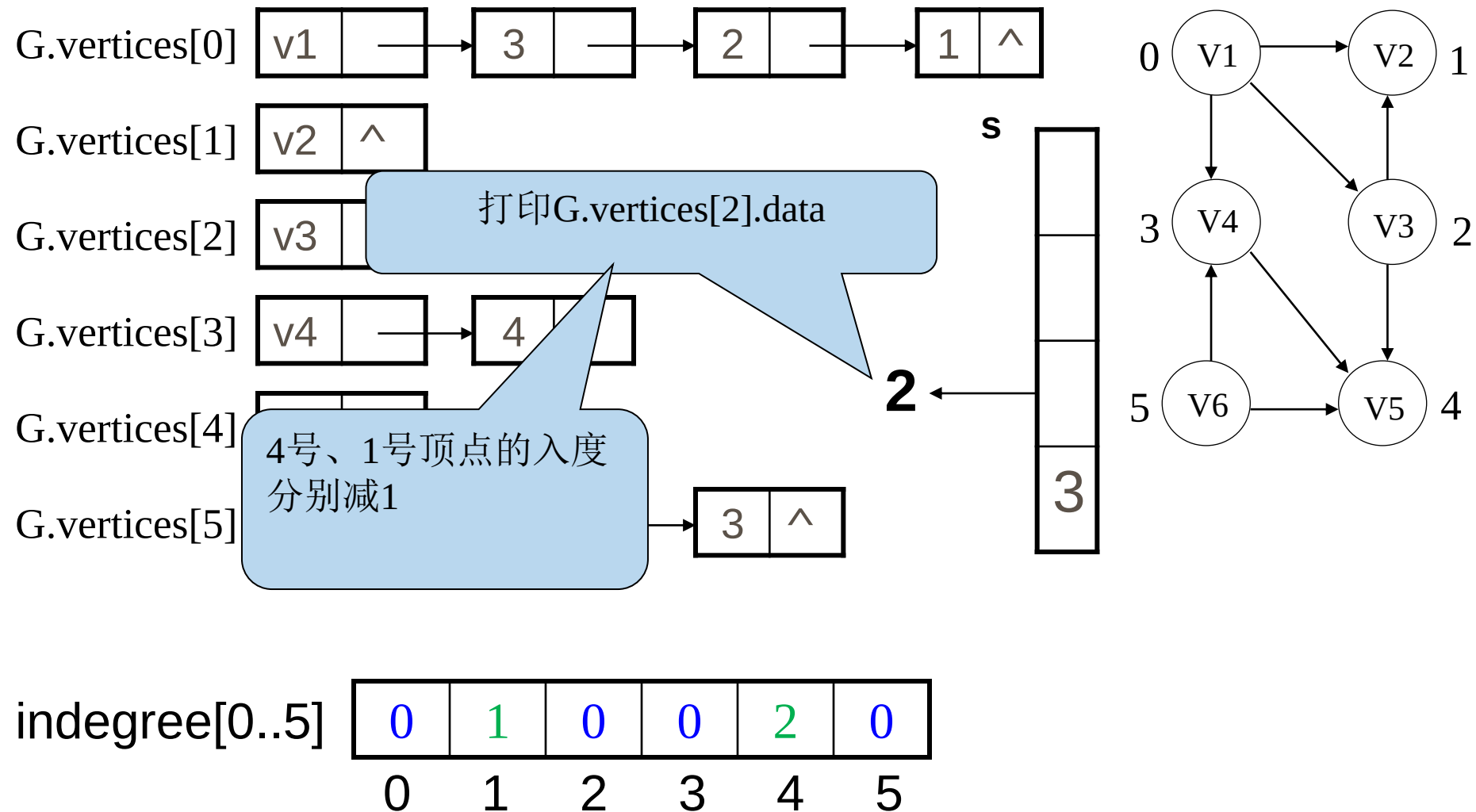
7.5.1 拓扑排序



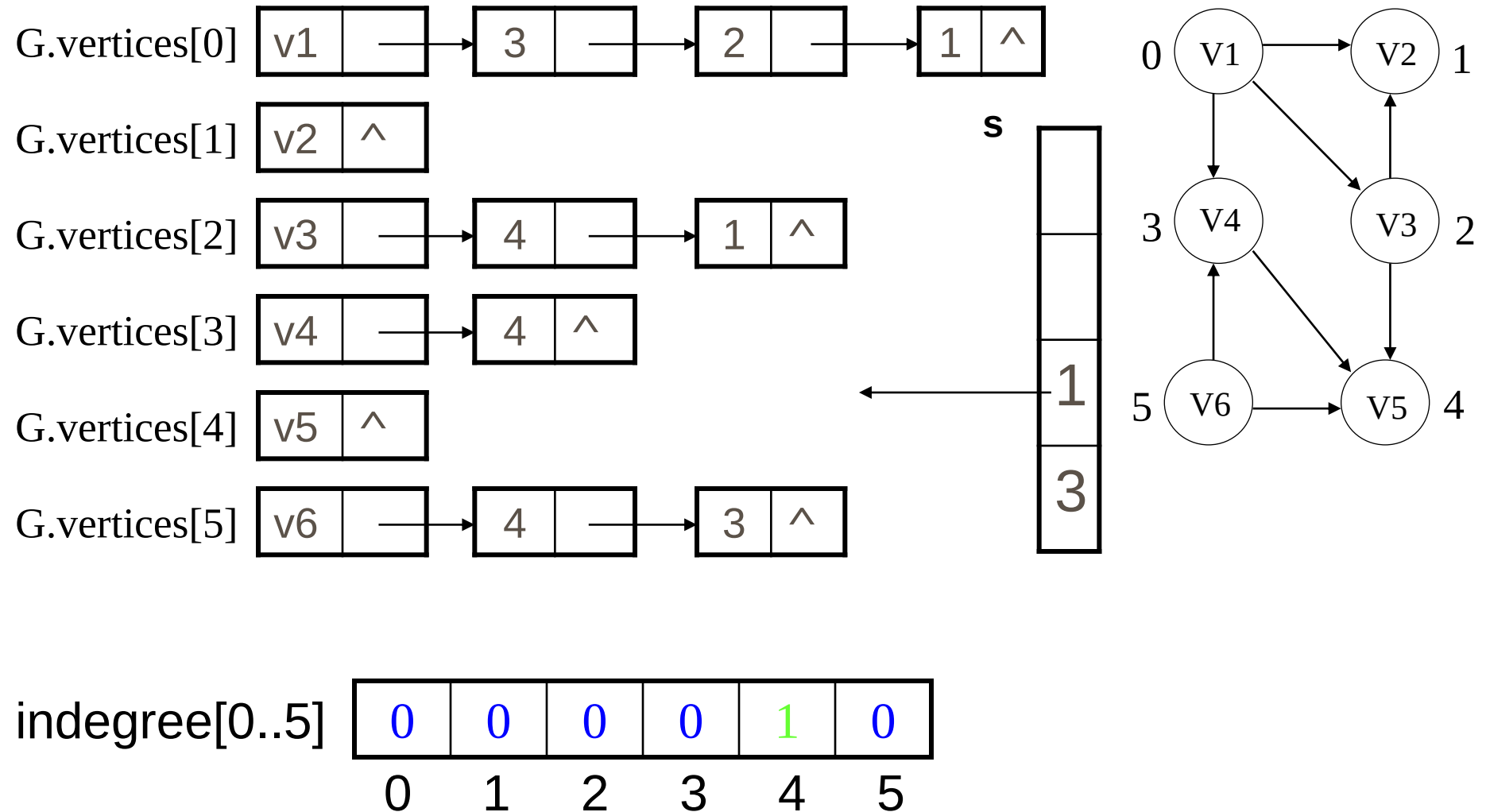
7.5.1 拓扑排序



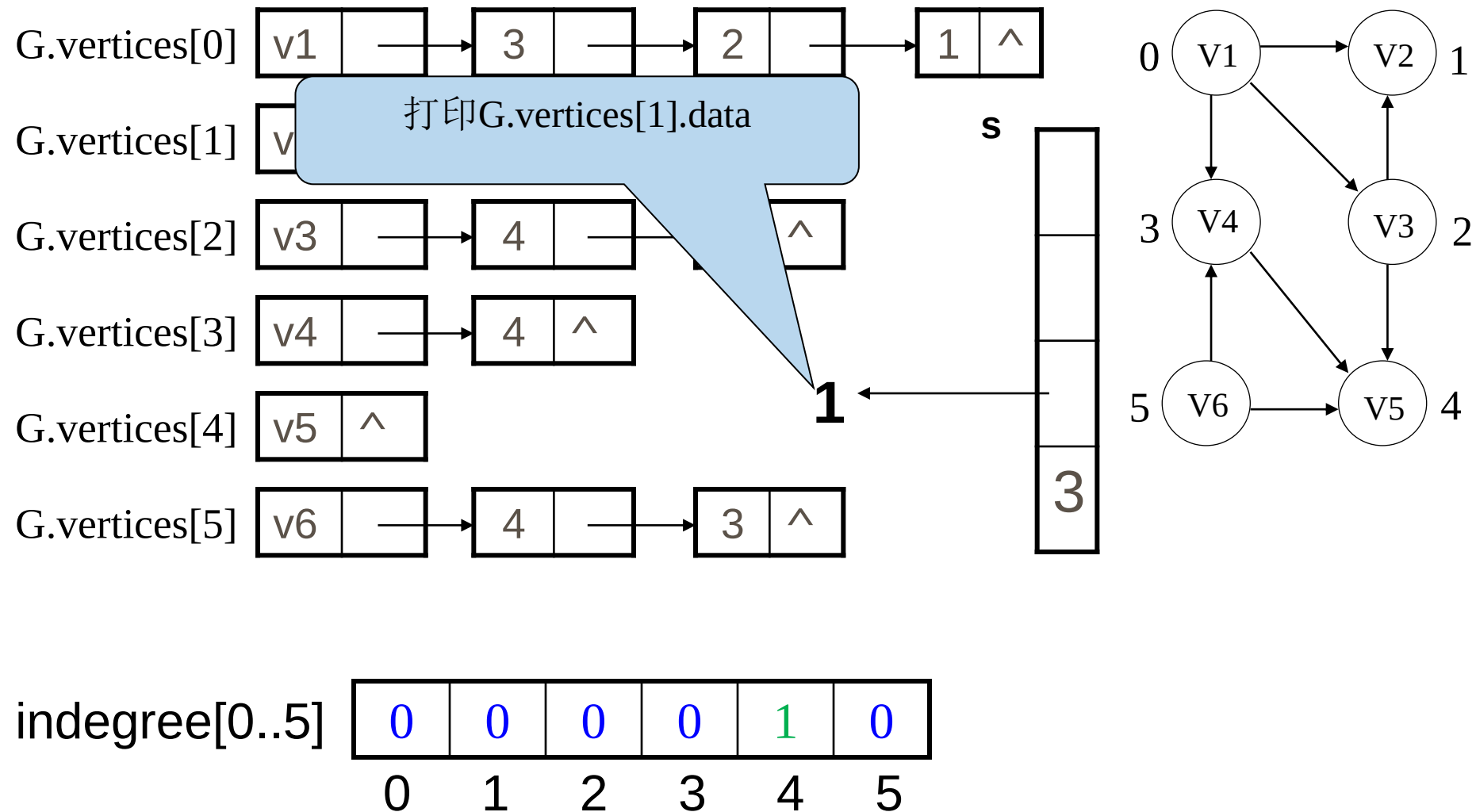
7.5.1 拓扑排序



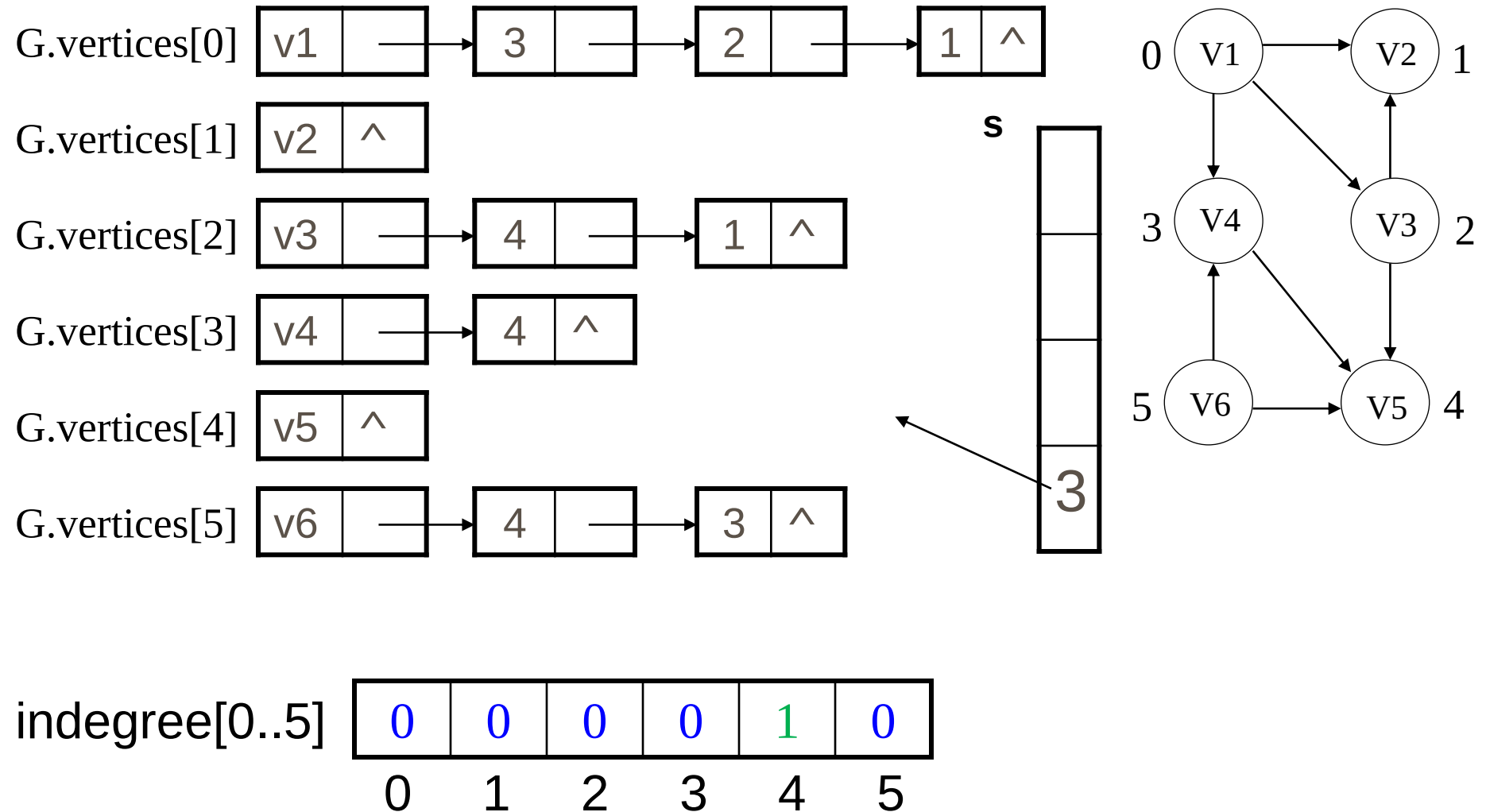
7.5.1 拓扑排序



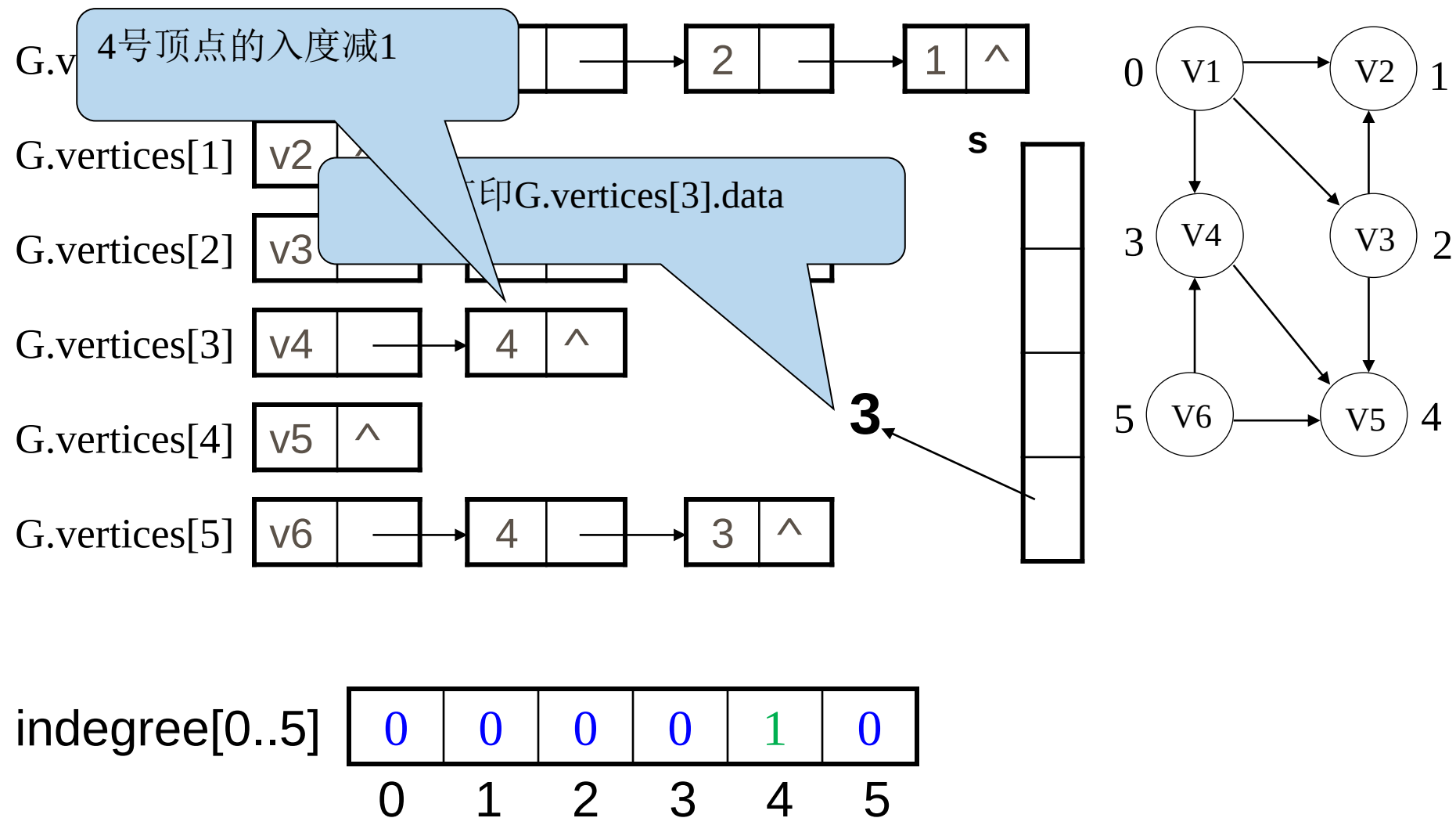
7.5.1 拓扑排序



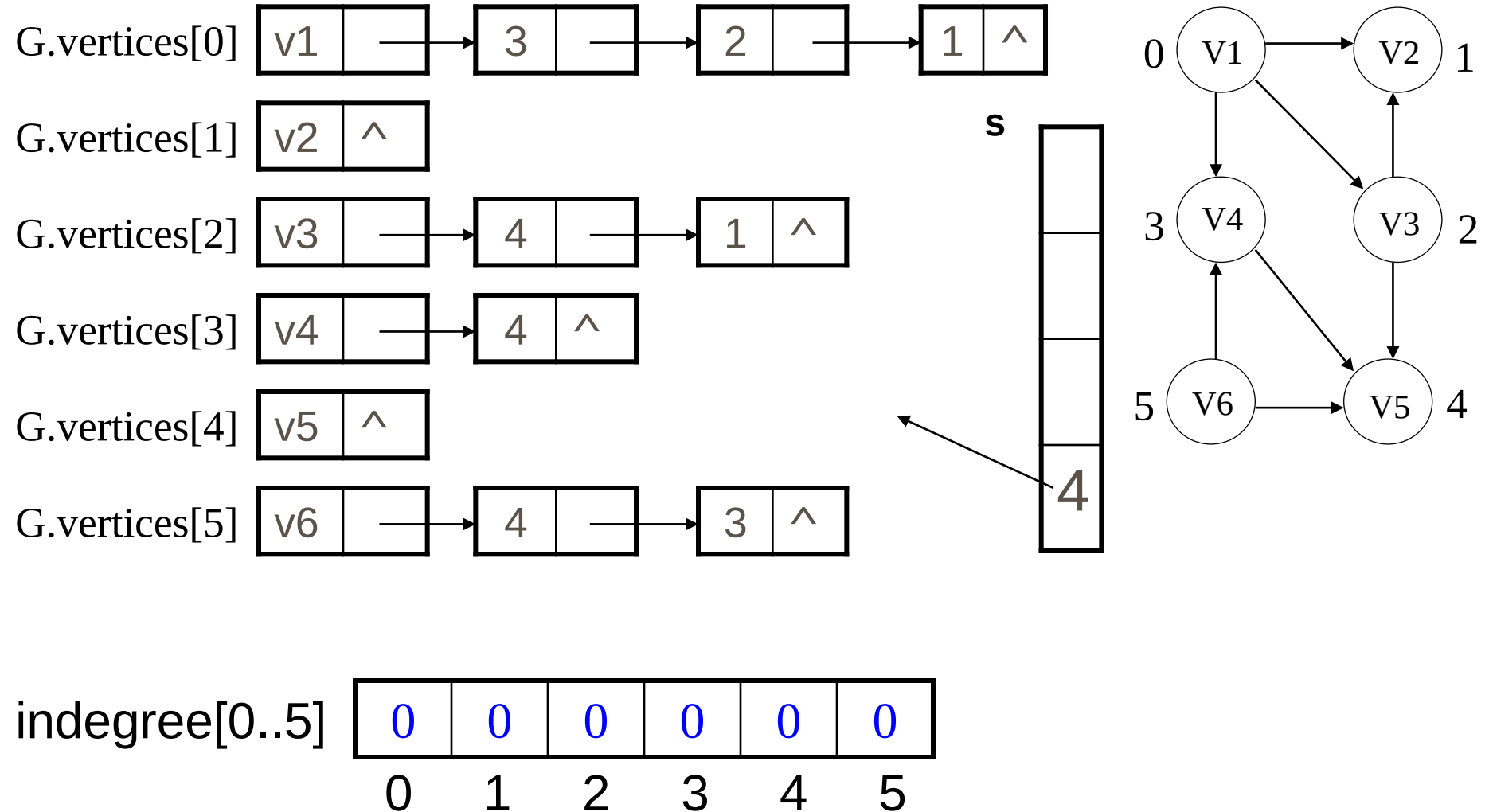
7.5.1 拓扑排序



7.5.1 拓扑排序

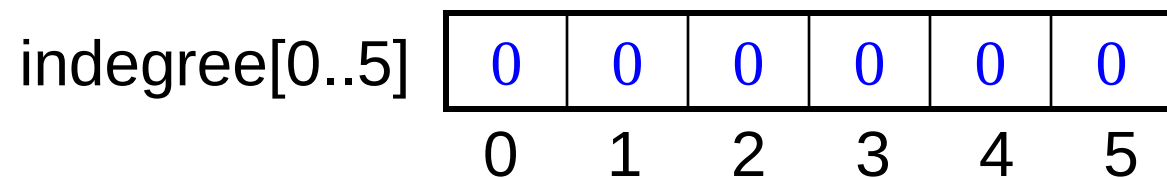
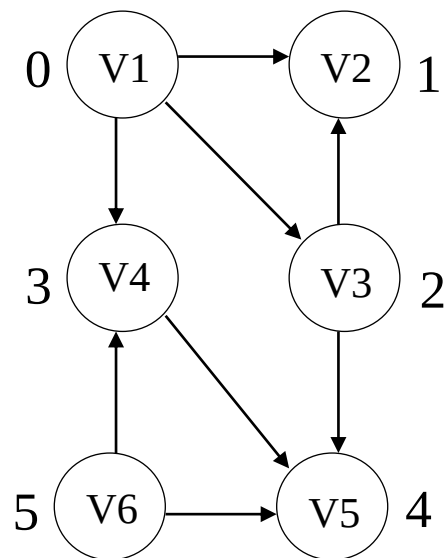
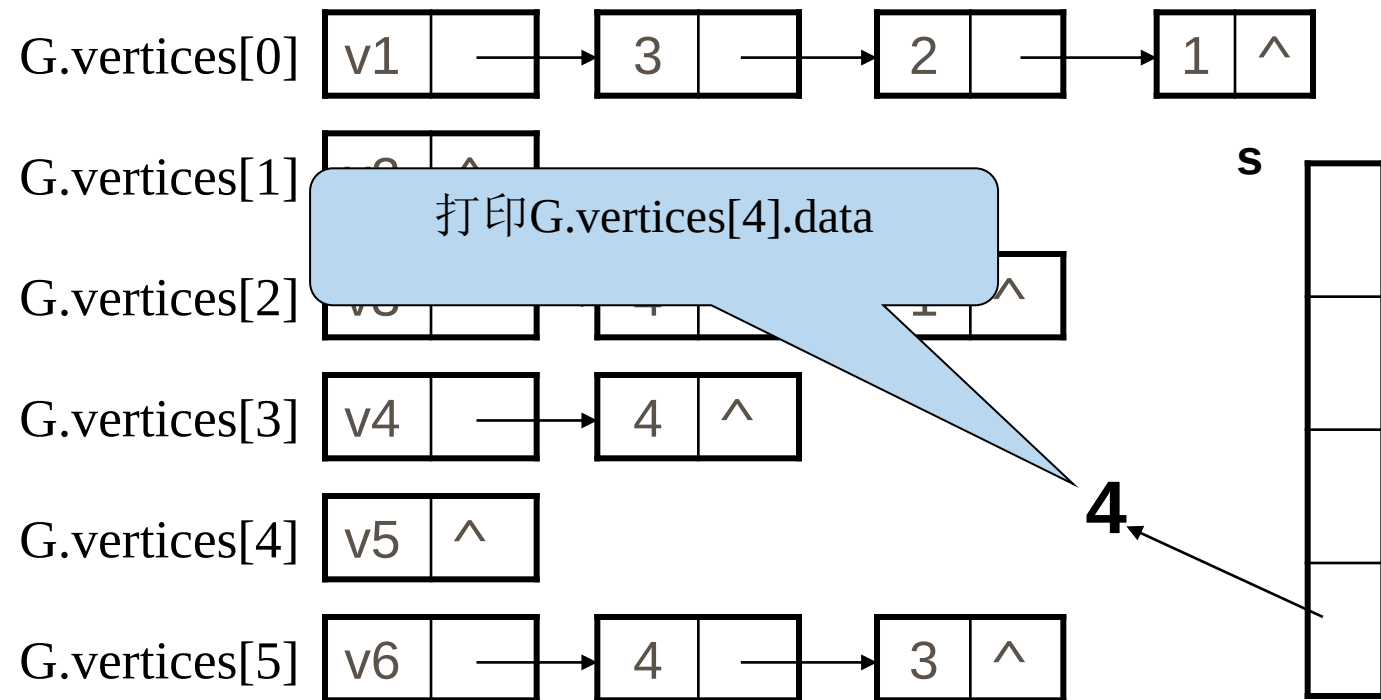


7.5.1 拓扑排序



7.5.1 拓扑排序

最后输出的拓扑序列为: $v_6v_1v_3v_2v_4v_5$



7.5.1 拓扑排序

Status Topological_Sort(ALGraph G){

//采用邻接表存储结构。若G无回路，则输出拓扑序列并返回OK，否则ERROR

FindInDegree(G, indegree); //对各顶点求入度indegree[0..vexnum-1]

InitStack(S); //建零入度顶点栈

for(i=0; i<G.vexnum; ++i) if(!indegree[i]) Push(S, i) //入度为0者进栈

count=0; //对输出顶点计数

while (!StackEmpty(S)) {

Pop(S, i);

printf(i, G.vertices[i].data); ++count; //输出i号顶点并计数

for(p=G.vertices[i].firstarc; p; p=p->nextarc){

k=p->adivex; //对i号顶点的每个邻接点入度减1

if(! (--indegree[k])) Push(S, k); //若入度减为0，则入栈

}//for

}//while

if(count<G.vexnum) return ERROR; //该有向图有回路

else return OK;

}//TopologicalSort

7.5.1 拓扑排序

□ 拓扑排序

算法分析

求顶点的入度： $T(n) = O(e)$

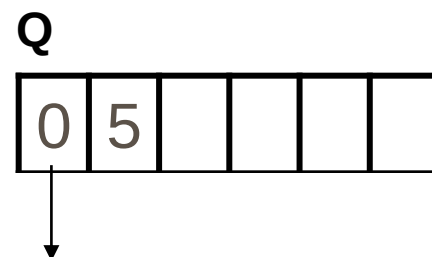
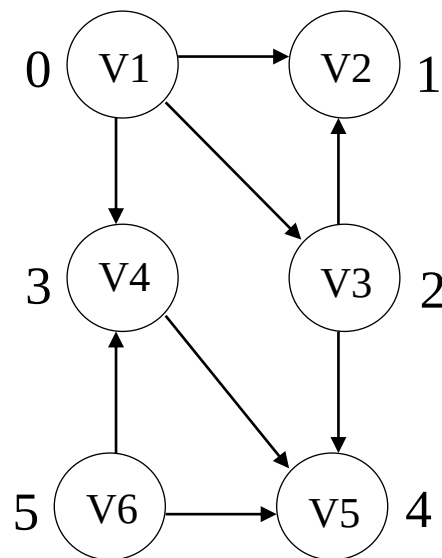
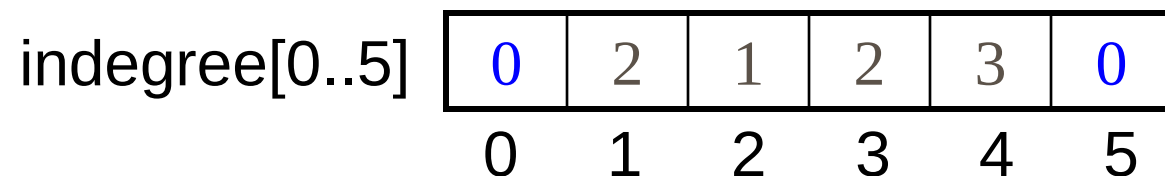
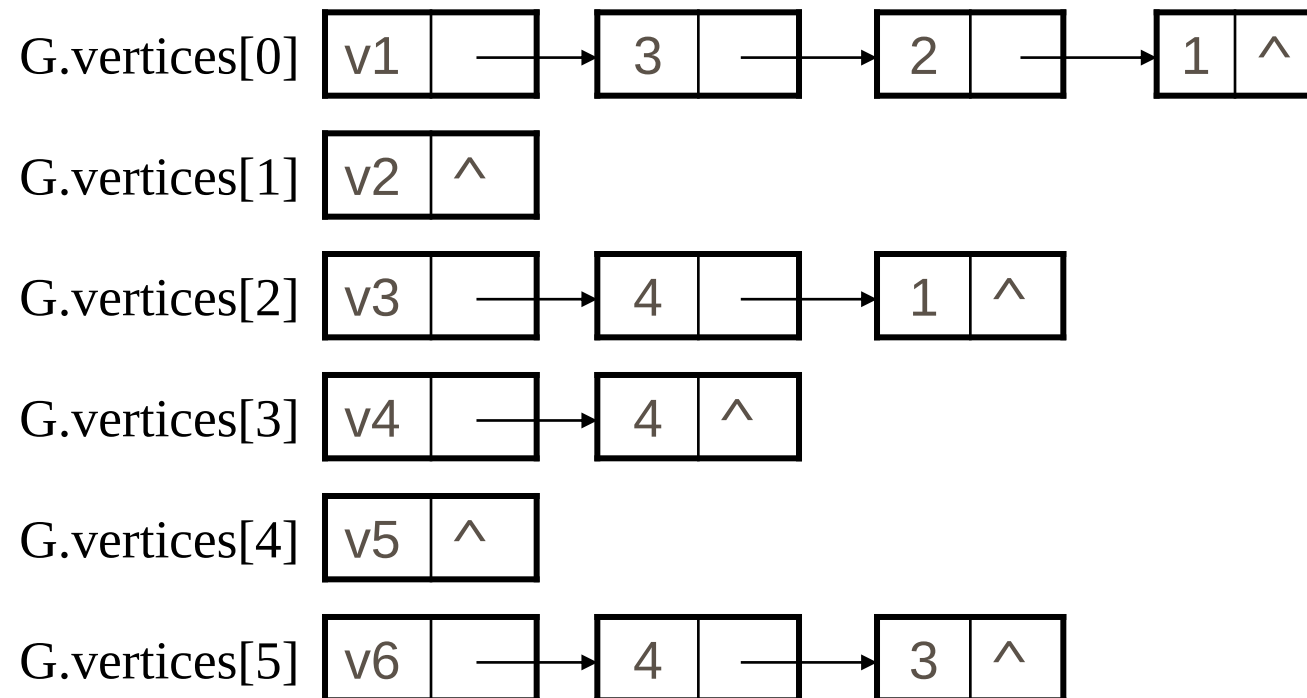
入度为0的顶点进栈： $T(n) = O(n)$

拓扑排序： $T(n) = O(n+e)$

总体时间复杂度： $T(n) = O(n+e)$

7.5.1 拓扑排序

采用队列存储入度为0的顶点

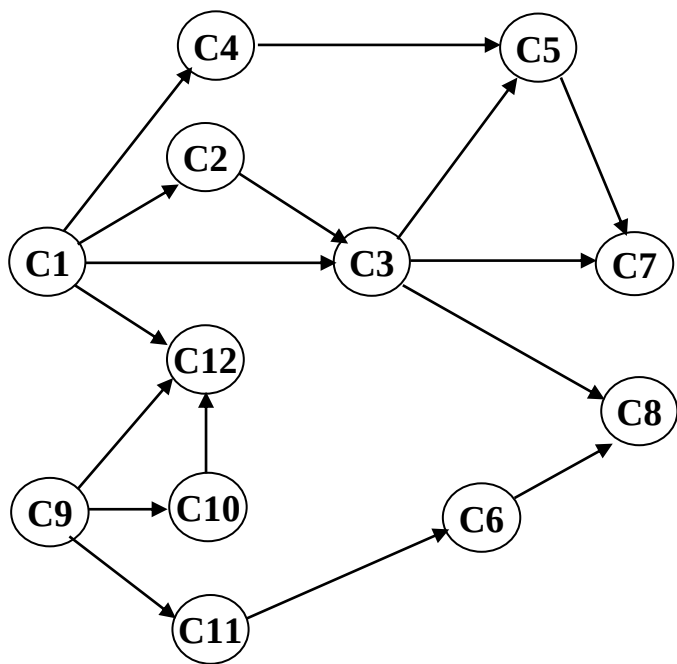


7.5.1 拓扑排序

```
Status TopologicalSort(ALGraph G) {
    front = back = 0; // 初始化队列指针
    for (i = 0; i < G.vexnum; i++) { // 将所有入度为0的节点加入队列
        if (!indegree[i]) queue[back++] = i;
    }
    count = 0; // 用于记录已经排序的节点数量
    while (front < back) { // 处理队列中的每个节点
        i = queue[front++]; // 从队列头部取出一个节点
        printf(i, G.vertices[i].data); ++count;
        for (p=G.vertices[i].firstarc; p; p=p->nextarc) {
            k=p->adivex;
            if (--indegree[k] == 0) // 减少目标节点的入度
                queue[back++] = k;
        }
    }
    if(count<G.vexnum) return ERROR;
    else return OK;
}
```

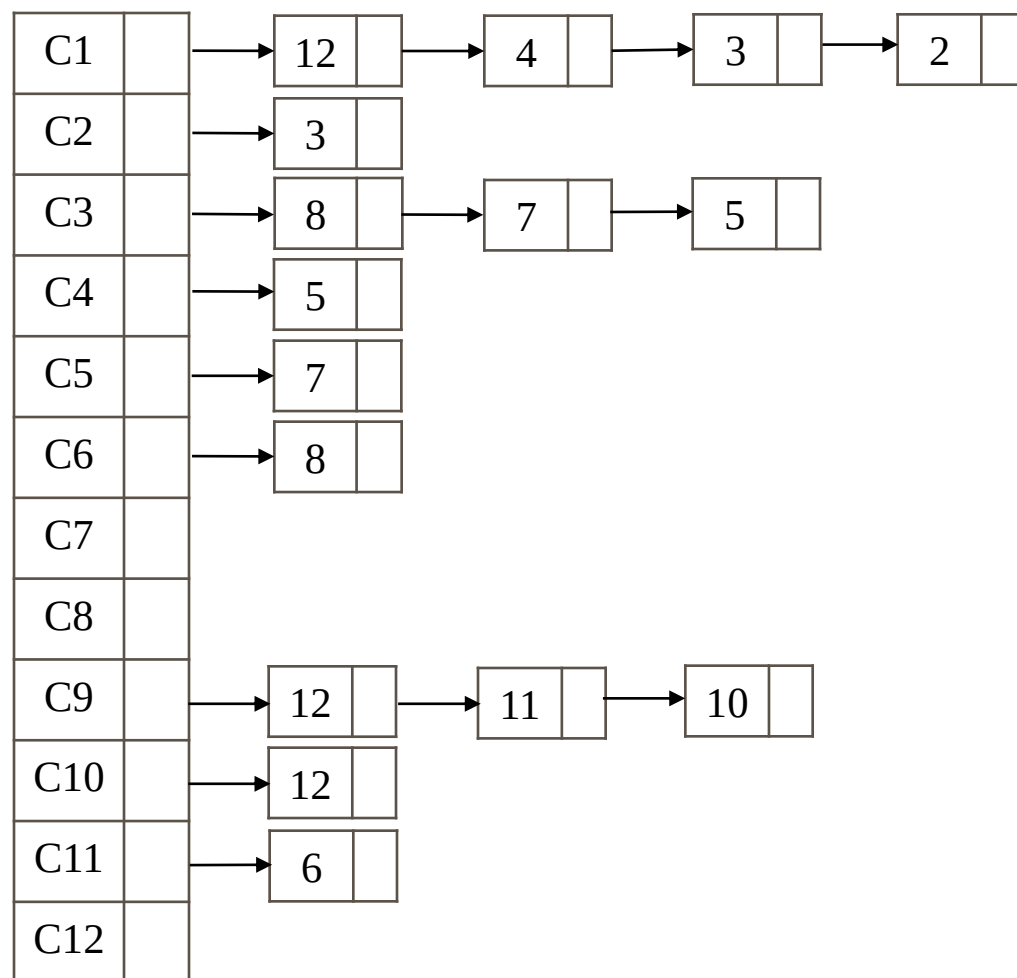
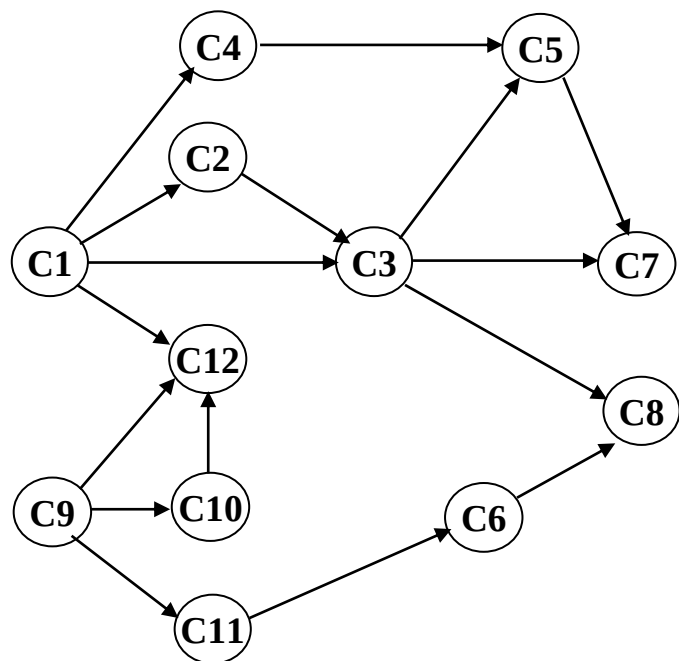
7.5.1 拓扑排序

例：定义节点的level: 若节点的入度为 0，则level为 0，若结点入度不为 0，则level为指向该结点的所有结点中最大的level +1。定义有向无环图 (DAG) 的高度: 所有结点中最大level 值。求每个节点的level，并计算DAG 的高度。



1. 定义一个队列，初始为空；定义level数组，记录每个点的level；
2. 将入度为0的点入队；
3. 出队队头元素(记为t)，考虑该元素的所有邻接点，若邻接点入度-1后为0，将该点(记为j)入队，同时更新其level数组: $\text{level}[j] = \max(\text{level}[j], \text{level}[t] + 1)$ ；
4. 重复3直到遍历完所有顶点；
5. level数组即为所求每个结点的level。

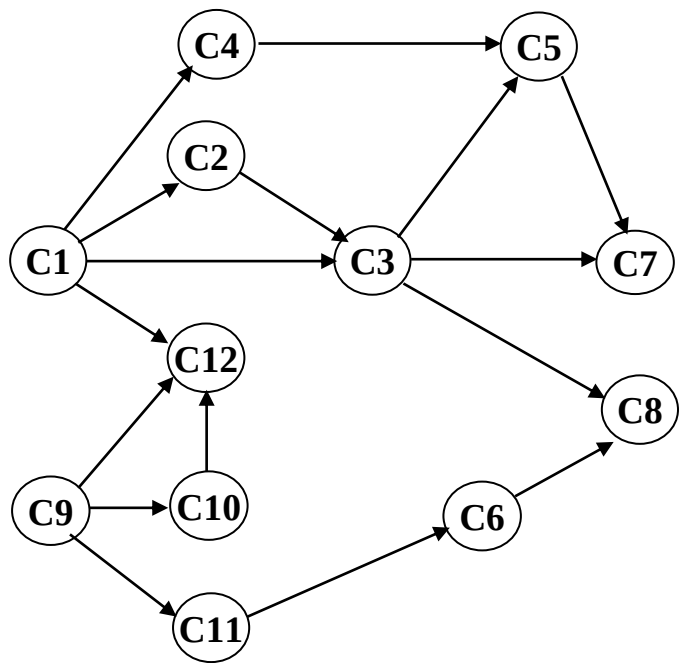
7.5.1 拓扑排序



1	2	3	4	5	6	7	8	9	10	11	12
0	1	2	1	2	1	2	2	0	1	1	3

7.5.1 拓扑排序

求逆拓扑有序序列



```
struct Node{  
    int id;  
    Node *next;  
} *head[N];
```

```
void RevTopologicalSort(){  
    for (int i = 1; i <= n; i++) {  
        if (!visited[i]) {  
            dfs(i);  
        }  
    }  
}  
  
void dfs(int u){  
    visited[u] = true  
    // 遍历当前结点的邻接点  
    for(Node *p = head[u]; p; p = p->next){  
        int j = p->id; // 邻接点编号  
        if (!visited[j]) // 若该点没访问过  
            dfs(j);  
    }  
    cout << u << ' ' ; // 退出递归时输出  
}
```

正在答疑
